

Nuh AI WhatsApp Bot

Complete Development & Deployment Guide

Project	sameer-ai-platform
Owner	Sameer Shaik
Company	N.S (Nadeem Shaik)
AI Name	Nuh 1.0
AI Engine	Groq API — llama-3.3-70b-versatile
Local Dev	Ubuntu 24 Laptop
Cloud	AWS EC2 — ap-southeast-2 (Sydney)
Instance	m7i-flex.large (2 vCPU, 8GB RAM)
Date	June 2026

This is the complete guide for building the Nuh AI WhatsApp Bot from scratch — from local development on a laptop, deploying to AWS EC2, connecting via VS Code Remote SSH, adding chat memory with Python, and all technical issues encountered with exact solutions.

Table of Contents

■■ PART A: LOCAL DEVELOPMENT — *Building the bot on your laptop*

A1 — WhatsApp Bot (Node.js + Baileys): *Bot setup, commands, human mode*

A2 — Spring Boot AI Gateway: *REST API connecting bot to AI*

A3 — Python FastAPI Service: *Mute/archive/block logic*

A4 — Ollama + Nuh Custom Model: *Local AI model creation*

A5 — Docker Compose Setup: *One-command local startup*

A6 — Tavily Live Search: *Internet search integration*

A7 — Groq API Integration: *Fast cloud AI instead of slow CPU*

A8 — Chat Memory Feature: *Per-user conversation history*

■■ PART B: AWS EC2 DEPLOYMENT — *Deploying to cloud*

B1 — EC2 Instance Setup: *Configuration and security*

B2 — SSH Connection: *Connecting to the server*

B3 — Server Setup: *Docker, Java, Ollama installation*

B4 — Project Transfer: *Uploading project to EC2*

B5 — Build and Deploy: *Running all containers on AWS*

■■ PART C: VS CODE REMOTE SSH — *Opening EC2 in VS Code*

C1 — Install Remote SSH Extension: *VS Code setup*

C2 — Configure SSH Host: *SSH config file*

C3 — Connect and Edit: *Working with EC2 in VS Code*

■■ PART D: ISSUES AND SOLUTIONS — *All technical problems fixed*

D1-D9 — 9 Issues Documented: *Exact errors and commands to fix*

PART A — Local Development (Lanton)

The entire project was first built and tested on a local Ubuntu laptop before deploying to AWS.

Project Structure

```
sameer-ai-platform/ ■■■ docker-compose.yml # One command to start everything ■■■ services/ ■  
■■■ ai-gateway/ # Spring Boot – AI REST API ■ ■ ■■■ src/main/java/ # ChatController.java ■ ■  
■■■ src/main/resources/ # application.properties ■ ■ ■■■ pom.xml ■ ■■■ whatsapp-bot/ # Node.js  
– WhatsApp handler ■ ■ ■■■ bot.js ■ ■ ■■■ auth/ # WhatsApp session files ■ ■■■ python-service/  
# FastAPI – block + memory logic ■ ■ ■■■ main.py ■ ■ ■■■ requirements.txt ■ ■■■ open-webui/ #  
Web dashboard (Docker image)
```

A1 — WhatsAnn Bot (Node.js + Baileys)

Setup

```
cd ~/Desktop/sameer-ai-platform/services/whatsapp-bot npm install
```

Key Features

Feature	How it works
Auto AI replies	Every DM goes to Nuh via Spring :8080
Groups ignored	All group messages silently skipped
Human mode	Say "I want to chat with Sameer" — bot goes silent
Wake AI	Say "Hey Nuh, question" — AI resumes
Mute/Archive	!mute jid — bot silent for that user
Chat memory	All conversations remembered per user
!forget	User clears their own chat history
Greet returning users	Hi shows different message if user has history

Bot Commands

Command	Description
!help	Show all commands
!ask <question>	Ask AI directly
!ping	Check bot speed
!time	Show current time
!info	Bot info and stats
!memory	Show how many messages remembered
!forget	Clear your chat history with Nuh
!mute <jid>	Stop bot from replying to a user
!unmute <jid>	Resume bot replies for a user
!archive <jid>	Archive (silence) a user
!unarchive <jid>	Resume archived user
!blocked	List all muted/archived users

Start bot locally

```
cd ~/Desktop/sameer-ai-platform/services/whatsapp-bot node bot.js
```

A2 — Spring Boot AI Gateway

Spring Boot acts as the bridge between the WhatsApp bot and the AI engine.

Endpoints

Endpoint	Method	Purpose
GET /api/ai/chat?message=...	GET	Simple AI chat (no memory)
POST /api/ai/chat/memory	POST	AI chat WITH conversation history

application.properties (Groq config)

```
spring.application.name=ai-gateway server.port=8080 # Groq API
spring.ai.openai.api-key=YOUR_GROQ_KEY spring.ai.openai.base-url=https://api.groq.com/openai/v1
spring.ai.openai.chat.options.model=llama-3.3-70b-versatile
spring.ai.openai.chat.options.temperature=0.7 # Tavily search
spring.ai.tavily.api-key=YOUR_TAVILY_KEY
```

Build

```
cd ~/Desktop/sameer-ai-platform/services/ai-gateway mvn clean package -DskipTests -q
```

A3 — Python FastAPI Service (Block + Memory)

Python handles two responsibilities: blocking muted/archived users AND storing chat memory per user.

API Endpoints

Endpoint	Method	Purpose
GET /should-respond?jid=...	GET	Check if bot should reply to this user
POST /mute	POST	Mute a user (bot goes silent)
DELETE /mute	DELETE	Unmute a user
POST /archive	POST	Archive a user
DELETE /archive	DELETE	Unarchive a user
GET /blocked	GET	List all muted/archived users
GET /memory/{jid}	GET	Get full chat history for a user
POST /memory	POST	Save a message to user history
DELETE /memory/{jid}	DELETE	Clear history for one user (!forget)
DELETE /memory	DELETE	Clear ALL users history (admin)
GET /memory	GET	List all users with history
GET /health	GET	Health check + stats

Memory Settings

```
MAX_HISTORY = 20 # Last 20 messages per user stored BLOCKED_FILE = 'blocked.json' # Persists muted/archived MEMORY_FILE = 'memory.json' # Persists chat history
```

Run locally

```
pip install fastapi uvicorn pydantic uvicorn main:app --host 0.0.0.0 --port 8000
```

A4 — Ollama + Nuh Custom Model

Install Ollama

```
curl -fsSL https://ollama.com/install.sh | sh
```

Pull base model

```
ollama pull qwen2.5:1.5b
```

Create Nuh Modelfile

```
cat > /tmp/Modelfile << 'EOF' FROM qwen2.5:1.5b SYSTEM "" You are Nuh, an AI assistant. Version: Nuh 1.0 Created by: Sameer Shaik Company: N.S Keep replies short. Plain text only. "" PARAMETER temperature 0.7 PARAMETER top_p 0.9 EOF
```

Build and verify

```
ollama create Nuh -f /tmp/Modelfile ollama list
```

✓ Nuh:latest 986MB created successfully

A5 — Docker Compose (One-Command Start)

docker-compose.yml key services

Service	Image/Build	Port	Purpose
ai-gateway	Build from ./services/ai-gateway	8080	Spring Boot AI API
whatsapp-bot	Build from ./services/whatsapp-bot	—	Node.js WhatsApp
python-service	Build from ./services/python-service	8000	FastAPI block+memory
open-webui	ghcr.io/open-webui/open-webui:main	3000	Web dashboard

Start/Stop commands

```
# Start all cd ~/Desktop/sameer-ai-platform docker compose up -d # Stop all docker compose down # Check status docker compose ps # View logs docker compose logs -f whatsapp-bot
```

A6 — Tavily Live Internet Search

Tavily gives Nuh the ability to search the internet for real-time information.

Get free API key

Go to <https://app.tavily.com> → Sign up → Create API Key

Keywords that trigger search

```
news, today, latest, current, weather, price, score, ipl, match, cricket, election, rupee,
sensex, 2026, live, who won, stock
```

How it works in Spring

```
// If keyword found → call Tavily → return answer directly // If no result → fall back to Groq
AI private boolean needsSearch(String msg) { String m = msg.toLowerCase(); return
m.contains("news") || m.contains("today") || ... }
```

A7 — Groq API Integration

Groq provides free, fast AI inference. Replaced slow CPU-based Ollama for production use.

Setting	Value
API URL	https://api.groq.com/openai/v1
Model	llama-3.3-70b-versatile
Free tier	Yes — generous free limits
Response time	1-3 seconds (vs 60-120s for CPU Ollama)
Spring AI starter	spring-ai-starter-model-openai (compatible)

Get free API key

Go to <https://console.groq.com> → Sign up → API Keys → Create API Key

A8 — Chat Memory Feature

Each user gets their own conversation history stored in Python. Nuh remembers context across messages.

Memory flow

```
User message → bot.js gets history from Python GET /memory/{jid} → sends history + new message
to Spring POST /api/ai/chat/memory → Spring builds message list with history → Groq replies with
full context → bot.js saves reply to Python POST /memory → sends reply to WhatsApp
```

!forget command flow

```
User types: !forget → bot.js calls DELETE /memory/{jid} → Python deletes all messages for that
user → Bot confirms: Memory cleared!
```

■ Note: Max 20 messages stored per user. Older messages auto-deleted when limit reached.

PART B — AWS EC2 Deployment

B1 — EC2 Instance Setup

Setting	Value	Reason
AMI	Ubuntu 26.04 LTS	Stable, modern
Instance	m7i-flex.large	2 vCPU 8GB — enough for all containers
Storage	30 GiB gp3 SSD	Fast, cheap (~\$2.40/month)
Key Pair	sameer-nuh-key.pem (RSA)	SSH access — save safely!
Purchasing	On-Demand	Pay per hour, stop anytime
Termination Protection	Enabled	Prevent accidental deletion

Security Group Rules

Port	Source	Purpose
22	My IP only	SSH — admin access
8080	My IP only	Spring AI Gateway
8000	My IP only	Python service
3000	My IP only	Open WebUI

Warning: Always use My IP for SSH — never 0.0.0.0/0 (Anywhere)!

B2 — SSH Connection

EC2 Public IP: 3.104.66.117

```
# Fix key permissions (run once) chmod 400 ~/Downloads/sameer-nuh-key.pem # Connect ssh -i ~/Downloads/sameer-nuh-key.pem ubuntu@3.104.66.117
```

✓ ubuntu@ip-172-31-39-34:~\$ — Connected!

B3 — Server Setup on EC2

Update system

```
sudo apt update && sudo apt upgrade -y sudo reboot # apply new kernel
```

Install Docker (official repo)

```
sudo apt install ca-certificates curl -y sudo install -m 0755 -d /etc/apt/keyrings sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc sudo chmod a+r /etc/apt/keyrings/docker.asc echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] \ https://download.docker.com/linux/ubuntu noble stable" | \ sudo tee /etc/apt/sources.list.d/docker.list > /dev/null sudo apt update && sudo apt install docker-ce docker-ce-cli containerd.io docker-compose-plugin -y sudo usermod -aG docker ubuntu && newgrp docker
```

Install Java + Maven

```
sudo apt install openjdk-21-jdk maven -y
```

Install Ollama + configure for Docker

```
curl -fsSL https://ollama.com/install.sh | sh sudo mkdir -p /etc/systemd/system/ollama.service.d sudo tee /etc/systemd/system/ollama.service.d/override.conf << 'EOF' [Service] Environment="OLLAMA_HOST=0.0.0.0" EOF sudo systemctl daemon-reload && sudo systemctl restart ollama
```

Create Nuh model on EC2

```
ollama pull qwen2.5:1.5b ollama create Nuh -f /tmp/Modelfile ollama list
```

B4 — Project Transfer to EC2

Zip on laptop (excludes node_modules)

```
cd ~/Desktop zip -r sameer-ai-platform.zip sameer-ai-platform \ --exclude '*/node_modules/*' --exclude '*/.venv/*'
```

Transfer zip

```
scp -i ~/Downloads/sameer-nuh-key.pem \ ~/Desktop/sameer-ai-platform.zip ubuntu@3.104.66.117:~/
```

Unzip on EC2

```
sudo apt install unzip -y unzip -o ~/sameer-ai-platform.zip -d ~/ ls ~/sameer-ai-platform/
```

B5 — Build and Deploy on AWS

Build Spring jar on EC2

```
cd ~/sameer-ai-platform/services/ai-gateway mvn clean package -DskipTests -q echo "Build: $?"
```

Start all containers

```
cd ~/sameer-ai-platform docker compose up -d docker compose ps
```

Test

```
curl http://localhost:8080/api/ai/chat?message=who+are+you curl http://localhost:8000/health
```

✓ I am Nuh, an AI assistant created by Sameer Shaik from N.S.

PART C — VS Code Remote SSH (Edit EC2 Like Local)

VS Code Remote SSH lets you open your EC2 project directly in VS Code — edit files, run terminals, manage Docker, all without using a separate SSH terminal.

C1 — Install Remote SSH Extension

1. Open VS Code on your laptop
2. Press **Ctrl+Shift+X** to open Extensions panel
3. Search: **Remote - SSH**
4. Install the extension by **Microsoft**
5. Restart VS Code

✓ Remote Explorer icon appears in left sidebar

C2 — Configure SSH Host

1. Press **Ctrl+Shift+P**
2. Type: **Remote-SSH: Open SSH Configuration File**
3. Select: **/home/sameer-shaik/.ssh/config**
4. **Select all (Ctrl+A) and delete everything**
5. Paste exactly this — nothing else:

```
Host sameer-ec2 HostName 3.104.66.117 User ubuntu IdentityFile ~/Downloads/sameer-nuh-key.pem
```

6. Save with **Ctrl+S**

■ Warning: Do NOT add any other content to this file — no C++ settings, nothing else!

C3 — Connect to EC2 and Edit Files

1. Press **Ctrl+Shift+P**
2. Type: **Remote-SSH: Connect to Host**
3. Select **sameer-ec2** from the list
4. New VS Code window opens — it installs VS Code Server on EC2 (first time ~2 mins)
5. Click **File** → **Open Folder**
6. Type: **/home/ubuntu/sameer-ai-platform** → OK
7. Press **Ctrl+`** to open terminal inside VS Code

✓ Your full EC2 project is now open in VS Code — edit any file directly!

What you can do from VS Code:

Action	How
Edit files	Click any file in Explorer panel — opens in editor
Run commands	Ctrl+` opens terminal connected to EC2
Rebuild after changes	Terminal: <code>cd ~/sameer-ai-platform && docker compose up -d --build</code>
View logs	Terminal: <code>docker compose logs -f whatsapp-bot</code>
Check containers	Terminal: <code>docker compose ps</code>

Stop saving money	Terminal: docker compose down (or stop EC2 from AWS Console)
-------------------	--

■ Note: VS Code bottom-left corner shows SSH: sameer-ec2 when connected to EC2

PART D — All Technical Issues and Solutions

Every issue encountered during development and deployment — with exact error messages and fix commands.

ISSUE 1 — docker-compose-plugin not found (EC2 Ubuntu 26.04)

```
&#10060; ERROR: sudo apt install docker-compose-plugin -y Error: Unable to locate package docker-compose-plugin
```

Ubuntu 26.04 default apt repo does not include docker-compose-plugin. Must add Docker's official repo.

```
&#10004; FIX: sudo apt install ca-certificates curl -y sudo install -m 0755 -d /etc/apt/keyrings  
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc echo  
"deb [...] https://download.docker.com/linux/ubuntu noble stable" | \ sudo tee  
/etc/apt/sources.list.d/docker.list > /dev/null sudo apt update && sudo apt install docker-ce  
docker-ce-cli containerd.io docker-compose-plugin -y
```

ISSUE 2 — SCP broken pipe (thousands of WhatsApp auth files)

```
&#10060; ERROR: scp -r ~/Desktop/sameer-ai-platform ubuntu@3.104.66.117:~/ client_loop: send  
disconnect: Broken pipe scp: Connection closed
```

The auth/ folder has thousands of tiny JSON files. Transferring file-by-file times out SSH connection.

```
&#10004; FIX: # Zip first on laptop (excludes node_modules) cd ~/Desktop zip -r  
sameer-ai-platform.zip sameer-ai-platform \ --exclude '*/node_modules/*' --exclude '*/.venv/*' #  
Transfer single zip scp -i ~/Downloads/sameer-nuh-key.pem \ ~/Desktop/sameer-ai-platform.zip  
ubuntu@3.104.66.117:~/ # Unzip on EC2 (use -o to skip per-file confirmation) unzip -o  
~/sameer-ai-platform.zip -d ~/
```

ISSUE 3 — Spring cannot reach Ollama (host.docker.internal fails on EC2)

```
&#10060; ERROR: curl http://localhost:8080/api/ai/chat?message=hello  
{"status":500,"error":"Internal Server Error"}
```

On EC2 Linux, host.docker.internal does not resolve. Docker containers need the actual bridge gateway IP.

```
&#10004; FIX: # Find Docker network gateway IP docker network inspect sameer-ai-platform_default |  
grep Gateway # Output: "Gateway": "172.18.0.1" # Verify Ollama reachable curl  
http://172.18.0.1:11434/api/tags # Update docker-compose.yml sed -i  
's|http://host.docker.internal:11434|http://172.18.0.1:11434|g' \  
~/sameer-ai-platform/docker-compose.yml # Update application.properties sed -i  
's|http://localhost:11434|http://172.18.0.1:11434|g' \  
~/sameer-ai-platform/services/ai-gateway/src/main/resources/application.properties
```

ISSUE 4 — AI takes 60-120 seconds (CPU-only Ollama)

```
&#10060; ERROR: # WhatsApp users getting: AI server offline # curl takes 60-120 seconds to respond  
# WARNING: No NVIDIA/AMD GPU detected. Ollama will run in CPU-only mode.
```

m7i-flex.large has no GPU. CPU-only inference is too slow for real-time WhatsApp. Switched to Groq API (free, 1-3s).

```
&#10004; FIX: # 1. Update pom.xml – replace ollama with openai starter sed -i  
's|spring-ai-starter-model-ollama|spring-ai-starter-model-openai|g' \  
~/sameer-ai-platform/services/ai-gateway/pom.xml # 2. Update application.properties cat >  
.../application.properties << 'EOF' spring.ai.openai.api-key=YOUR_GROQ_KEY  
spring.ai.openai.base-url=https://api.groq.com/openai/v1  
spring.ai.openai.chat.options.model=llama-3.3-70b-versatile EOF # 3. Rebuild and restart mvn clean  
package -DskipTests -q && docker compose up -d --build
```

ISSUE 5 — 404 Wrong Groq API URL

```
&#10060; ERROR: 404: Unknown request URL: POST /openai/chat/completions Please check the URL for  
typos
```

base-url was https://api.groq.com/openai — Spring AI appends /chat/completions making /openai/chat/completions. Groq needs /openai/v1/chat/completions.

```
&#10004; FIX: sed -i 's|https://api.groq.com/openai|https://api.groq.com/openai/v1|g' \  
~/sameer-ai-platform/services/ai-gateway/src/main/resources/application.properties # Correct  
value: # spring.ai.openai.base-url=https://api.groq.com/openai/v1
```

ISSUE 6 — Groq Model Decommissioned

```
&#10060; ERROR: 400: The model 'llama3-8b-8192' has been decommissioned and is no longer  
supported. Please refer to https://console.groq.com/docs/deprecations
```

llama3-8b-8192 was deprecated by Groq. Updated to current model.

```
&#10004; FIX: sed -i 's|llama3-8b-8192|llama-3.3-70b-versatile|g' \  
~/sameer-ai-platform/services/ai-gateway/src/main/resources/application.properties
```

ISSUE 7 — Controller not found (404) after moving file

```
&#10060; ERROR: curl http://localhost:8080/api/ai/chat?message=hello {"status":404,"error":"Not  
Found"}
```

ChatController.java was placed in wrong folder (project root /controller/) instead of inside src/main/java/. Spring only scans src/main/java/.

```
&#10004; FIX: mkdir -p  
~/sameer-ai-platform/services/ai-gateway/src/main/java/com/sameer/ai_gateway/controller mv  
~/sameer-ai-platform/services/ai-gateway/controller/ChatController.java \  
~/sameer-ai-platform/services/ai-gateway/src/main/java/com/sameer/ai_gateway/controller/ #  
Rebuild mvn clean package -DskipTests -q
```

ISSUE 8 — VS Code SSH Config mixed with C++ settings

```
&#10060; ERROR: SSH config file contained C++ formatting settings: cpp_indent_braces=false  
cpp_space_after_comma=true ...
```

SSH config file got corrupted with extra C++ formatting content. VS Code could not parse the SSH host.

```
&#10004; FIX: # Open SSH config # Ctrl+Shift+P -> Remote-SSH: Open SSH Configuration File # Press  
Ctrl+A to select all, Delete, then paste ONLY: Host sameer-ec2 HostName 3.104.66.117 User ubuntu  
IdentityFile ~/Downloads/sameer-nuh-key.pem # Save with Ctrl+S
```

ISSUE 9 — application.properties missing (Spring 500 on first run)

```
&#10060; ERROR: curl http://localhost:8080/api/ai/chat?message=hello  
{"status":500,"error":"Internal Server Error"}
```

No application.properties file existed. Spring AI did not know where Ollama was running.

```
&#10004; FIX: mkdir -p ~/sameer-ai-platform/services/ai-gateway/src/main/resources cat >  
.../application.properties << 'EOF' spring.application.name=ai-gateway server.port=8080  
spring.ai.ollama.base-url=http://localhost:11434 spring.ai.ollama.chat.model=Nuh EOF
```

Quick Reference — Daily Commands

Task	Command
Start everything (EC2)	<code>cd ~/sameer-ai-platform && docker compose up -d</code>
Stop everything	<code>docker compose down</code>
Check all containers	<code>docker compose ps</code>
View bot logs	<code>docker compose logs -f whatsapp-bot</code>
View all logs	<code>docker compose logs -f</code>
Test AI	<code>curl http://localhost:8080/api/ai/chat?message=hello</code>
Test Python	<code>curl http://localhost:8000/health</code>
Check memory users	<code>curl http://localhost:8000/memory</code>
SSH into EC2	<code>ssh -i ~/Downloads/sameer-nuh-key.pem ubuntu@3.104.66.117</code>
Rebuild after code change	<code>mvn clean package -DskipTests -q && docker compose up -d --build</code>
Stop EC2 (save money)	AWS Console -> Instances -> Stop

Cost Summary

State	Cost
EC2 Running	~\$0.12/hr (~\$86/month)
EC2 Stopped	FREE compute + ~\$2.40/month storage
Groq API	Free tier (generous limits)
Tavily Search	Free tier (1000 searches/month)